

THE RASPBERRY PI PROJECT

ROBOTIC ARM



Alberto Fasulo
Liceo Lugano 1
Matura Project (High School Graduation Project)
2021-2022
Computer Science - "Il lampone informatico"
Supervisor: Amedeo Mazzoleni

English translation for portfolio use. The technical content and development narrative of the original report have been preserved, with light editing for clarity and natural English.

Abstract

After choosing my Matura project topic without too much difficulty, I still had to decide what I actually wanted to build. Thanks to the performance and versatility of the Raspberry Pi - a compact single-board computer - there were many possibilities. Because I have always enjoyed hands-on projects, I decided to use the computer to control physical devices, similarly to what I had already done with Arduino. At the same time, I wanted to make full use of the Raspberry Pi's capabilities, so I set myself the goal of making the project remotely controllable from another device.

Even after narrowing the scope, there were still many possible ideas. In the end, however, the idea found me rather than the other way around. One day at lunch, some classmates and I were joking about a friend's project: he wanted to use Arduino to build a robotic arm that moved in sync with his own arm. Doing this with Arduino would have been quite demanding, and I realized that I could approach a similar idea with my project and control it remotely, for example from my phone. That became the concept for my Matura project, and my supervisor approved it.

With the idea chosen and approved, the next step was to turn it into a working robotic arm that could be controlled remotely.

Contents

1 Introduction

2 Mark I

- 2.1 Background
- 2.2 Materials
- 2.3 Construction

3 Mark II

- 3.1 Materials
- 3.2 Design and construction
- 3.3 Motion-control program
 - 3.3.1 Libraries
 - 3.3.2 Movement
 - 3.3.3 Mark II and its “nervousness”
 - 3.3.4 Movement 1.2
- 3.4 Object grabbing
 - 3.4.1 Distance-detection program
 - 3.4.2 Distance-based movement
 - 3.4.3 Trigonometry-based movement
- 3.5 Mark II gets a new look
 - 3.5.1 The new look 2.0

4 Remote control

- 4.1 Website control
 - 4.1.1 Saving the arm position
- 4.2 SSH connection

5 Conclusions

6 Bibliography

1 Introduction

When the proposed Matura project topics were uploaded to Moodle, I started browsing through them to find the one that suited me best. I liked several of the options, but one immediately caught my attention, probably because of its unusual and slightly playful title: “Il lampone informatico” (literally, “The Computing Raspberry”). I opened the description straight away and, after reading the presentation, selected it almost immediately as my first choice.

The main reason was my strong interest in computer science. The project was based on developing software with a Raspberry Pi, and in my free time I already enjoyed programming and building small “robots” using Arduino. One example was a pressure-rocket parachute release system. When I saw the opportunity to learn how to control physical devices with a similar but more powerful platform, using a different programming language and offering many more possibilities, I decided to take it.

2 Mark I

2.1 Background

The term “Mark” is commonly used to indicate the development stage or version of a machine. I use it throughout this report to document the progression of my project: the higher the number, the more advanced the prototype.

After learning the basics of Raspberry Pi programming, I decided to build a first prototype in order to understand how to control multiple servo motors from the same board.

2.2 Materials

The materials used for this first prototype were deliberately simple. Excluding cables and the Raspberry Pi itself, the main components were:

- 2 × SG90 servo motors
- LEGO components

The LEGO pieces formed the base structure to which the servos were attached. This first prototype was not really an arm; it was closer to a small rotating tower with a second servo on top. Its purpose was to help me become familiar with controlling servo motors through the Raspberry Pi.

Before continuing, it is useful to briefly explain what a servo motor is. A servo motor is an actuator designed to provide controlled angular or linear position, typically using a motor, gearing, feedback sensing and a control circuit. In practical terms, unlike a simple motor that mainly spins, a servo allows the program to command a specific angular position.

This makes servos particularly useful for robotics and automation. In my project, they allowed me to control the angle of the individual joints of the robotic arm, in a way comparable to bending a human elbow to a specific angle.

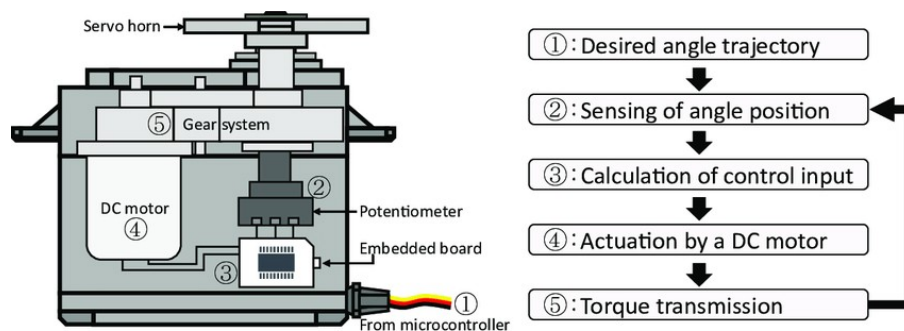


Figure 1 - Schematic representation of an RC servo motor

2.3 Construction

Returning to Mark I, the first prototype was a square-based lattice tower made from LEGO, approximately ten centimetres tall and five centimetres wide. A servo motor at the base rotated the entire structure horizontally and also acted as a support. At the top of the tower, I attached a second servo to a square LEGO platform with hot glue; this second motor represented the arm.

Both the construction and the programming of Mark I were relatively straightforward. Once I had learned how to control a simple LED and then servo motors through the GPIO pins, the prototype began to work reasonably well. The only significant issue was keyboard input. I initially planned to use Python's keyboard library, but it was not supported in the way I needed on the Raspberry Pi's Linux environment. I therefore replaced it with pygame, which allowed me to detect keyboard input successfully.

The two servo motors could then be moved with the computer keyboard using a simple program of only a few lines. Unfortunately, the source code for this first prototype was later lost after an accident damaged the Raspberry Pi's memory card. More than a finished robotic arm, Mark I was therefore my first practical introduction to programming and controlling physical hardware with the Raspberry Pi.



Figure 2 - Mark I lattice-tower prototype

3 Mark II

3.1 Materials

After becoming familiar with the new platform and understanding the basic control mechanisms, I started building my first full robotic-arm prototype. Mark I had shown me that the SG90 servos were not suitable for a larger and more capable arm.

There were two main reasons. First, their available torque was too low for the size of the arm I wanted to build. An SG90 that can lift roughly one kilogram at one centimetre from the shaft would only be able to lift a much smaller

load at a longer lever arm. Second, the physical design of the motor provided only one convenient rotating attachment point, which made it difficult to connect arm segments in the way I needed.

I therefore searched for a more appropriate servo and selected 25 kg RDS3225 robot servos. These had stronger torque and a mechanical layout better suited to a robotic arm. I bought six of them together with aluminium brackets designed for robotic assemblies.

3.2 Design and construction

When the new servos arrived, I first assembled and tested them individually to understand whether they behaved differently from the motors I had used before. I then began designing the arm and choosing suitable dimensions. I opted for two arm segments of different lengths: approximately 19 cm and 21 cm.

My first idea was to build the arm segments from LEGO, as I had done for Mark I, but this did not work well. I looked for alternatives and eventually chose wooden strips. I found pieces in the garage whose width and thickness were close to those of the servo brackets, cut them to the required lengths with a jigsaw, and began assembling Mark II.

For the first tests I used three servo motors and left the gripper aside. I connected the motors to a breadboard and then to the Raspberry Pi. Each servo had three wires: a positive supply wire, a ground wire and a signal wire. The GPIO pins on the Raspberry Pi made it possible not only to provide electrical signals to components but also to generate controllable signals for devices such as servos and to receive data from sensors.

The positive supply was connected through the breadboard to the 5 V supply, the ground to a ground pin, and each servo's signal wire to a controllable GPIO pin. I repeated this process for the first three servos. One practical problem remained: the arm could not stand securely on its own. I solved this very directly by drilling into the desk and fastening the servo brackets to its edge. Once all motors were connected and the arm was mechanically stable, I moved on to the software.

3.3 Motion-control program

After testing the first three motors, I installed two additional motors at the end of the arm to form a gripper. At this point I wrote the first complete program for controlling the arm.

3.3.1 Libraries

The first library I used to control the arm - and more specifically the motors - was RPi.GPIO. This library simplifies access to the Raspberry Pi's GPIO pins. After importing it, the program must define the pin-numbering convention and configure each pin as either an input or an output. Once that is done, the pins can be controlled through relatively little code.

Because the GPIO pins were driving servo motors, I also needed to generate a pulse-width-modulated signal at an appropriate frequency. In the first version of the program I used a 50 Hz PWM signal and controlled each servo through its duty cycle.

I also needed a way to decide which motor should move. As with Mark I, I wanted to use the keyboard. Since the keyboard library was not suitable for my setup, I used pygame. Although pygame is primarily designed for videogame development, it can process keyboard events very effectively. In my case, instead of moving a character made of pixels, those keyboard events moved a physical wooden arm.

```

1  import RPi.GPIO as GPIO
2  import time
3  import pygame
4  pygame.init()
5  clock = pygame.time.Clock()
6  pygame.display.set_mode((300, 300))
7  #inizializzazione pin
8  GPIO.setmode(GPIO.BCM)
9  GPIO.setup(17,GPIO.OUT)
10 GPIO.setup(23,GPIO.OUT)
11 GPIO.setup(24,GPIO.OUT)
12 GPIO.setup(25,GPIO.OUT)
13 GPIO.setup(8,GPIO.OUT)
14
15
16
17 GPIO.setwarnings(False)
18
19 servo1 = GPIO.PWM(17,50)
20 servo2 = GPIO.PWM(23,50)
21 servo3 = GPIO.PWM(24,50)
22 servo4 = GPIO.PWM(25,50)
23 servo5 = GPIO.PWM(8,50)
24

```

Figure 3 - Initial RPi.GPIO configuration and PWM setup

3.3.2 Movement

Once the libraries had been configured, I could write the movement logic. The keyboard-control code was conceptually simple: it was an expanded version of the program developed for Mark I, now adapted to several motors.

The movement function used one variable for each motor plus another variable to indicate which motor was currently selected. The keyboard handling was divided into three parts. First, number keys selected the motor to control. Second, an if statement activated the logic associated with the selected motor. Third, the left and right arrow keys increased or decreased that motor's control variable.

I also added limits so that the variables could never exceed values accepted by the servos. If a variable became too large or too small, it was reset to the relevant maximum or minimum. The same logic was repeated for each joint, with the main exception being the gripper: its two motors had to move together in opposite directions.

```

35 def tastiera():
36     b = 9.5
37     c = 9.5
38     d = 7
39     e = 9
40     f = 8
41     #variabile per la scelta del motore
42     stato = 1
43     running = True
44     #schifo di pygame iniziale
45     while running:
46         for event in pygame.event.get():
47             if event.type == pygame.QUIT:
48                 running = False
49                 pygame.quit()
50
51
52
53             if event.type == pygame.KEYDOWN:
54                 if event.key == pygame.K_1:
55                     stato = 1
56                     #movimento motore 1
57                     if stato == 1:
58                         if event.key == pygame.K_LEFT:
59                             b = b + 0.3
60                             if b > 12:
61                                 b = 12
62
63
64                         elif event.key == pygame.K_RIGHT:
65                             b = b - 0.3
66                             if b < 2:
67                                 b = 2

```

Figure 4 - Keyboard input and motor-selection logic

The movement variables then had to be converted into actual commands for the motors. In this first implementation, each PWM object was assigned to a GPIO pin and the `ChangeDutyCycle` command was used to send the updated duty-cycle value to the relevant servo.

```

117     servo1.ChangeDutyCycle(b)
118     servo2.ChangeDutyCycle(c)
119     servo3.ChangeDutyCycle(d)
120     servo4.ChangeDutyCycle(e)
121     servo5.ChangeDutyCycle(f)

```

Figure 5 - Updating the servo duty cycles

This led to the first of many debugging problems. When I pressed a key, the coordinate variable changed and the motor moved, but to move it again I had to release the key and press it again. The cause was that pygame was only processing movement when an event occurred. Once I modified the program so that pygame updated at short intervals, continuous movement worked correctly.

3.3.3 Mark II and its “nervousness”

The first major problem I encountered was what I called the arm’s “nervousness”. The code now allowed me to control each motor individually and even position the arm to grab objects, but whenever the servos were supposed to remain still, their shafts oscillated slightly from side to side. The arm was therefore never completely stable.

This behaviour is commonly known as servo “jitter”. It became one of the most difficult issues in the entire project. I spent much of the first month of the summer trying to solve it: reading documentation, moving from forum to forum, watching tutorials and, as a sign of how far the search had gone, even reaching the second page of Google results.

Different sources suggested different causes: an incorrect signal frequency, missing capacitors, unstable power delivery and several other possibilities. Eventually I decided to attack the problem at a different level and replace the software library I was using.

3.3.4 Movement 1.2

The alternative that looked most promising was gpiozero, another library designed to simplify GPIO control on the Raspberry Pi. In my view it provided a more convenient set of functions, including a Servo class specifically designed for servo-motor control.

After rewriting the program with gpiozero, the jitter was reduced but still present. While reading the documentation, however, I found PiGPIOFactory. Using pigpio as the pin factory provided much more stable timing for the servo signals, and in my setup it finally eliminated the jitter problem.

The overall program structure remained similar, but the motor setup became cleaner and the movement code was reorganized into a more readable form.

```

2  from gpiozero import Servo
3  import time
4  from gpiozero.pins.pigpio import PiGPIOFactory
5
6
7  import pygame
8  pygame.init()
9
10 factory= PiGPIOFactory()
11 servo1_pin = 27
12 servo2_pin = 12
13 servo3_pin = 24
14 servo4_pin = 25
15 servo5_pin = 8
16
17 servo1 = Servo(servo1_pin, pin_factory=factory)
18 servo2 = Servo(servo2_pin, pin_factory=factory)
19 servo3 = Servo(servo3_pin, pin_factory=factory)
20 servo4 = Servo(servo4_pin, pin_factory=factory)
21 servo5 = Servo(servo5_pin, pin_factory=factory)

```

Figure 6 - Servo configuration using gpiozero and PiGPIOFactory

3.4 Object grabbing

Once the manual movement program was working, I turned to the second major part of the project: remote control. My first idea was to use the CGI protocol to create a small web server and a dynamic page in Python. The goal was to build a web interface with HTML buttons that could control every motor on the arm.

This first attempt did not progress as planned because I had difficulties configuring Apache2 and connecting the Python and HTML files through CGI. After many attempts, including several reinstalls, I decided to pause that part until I could ask my supervisor for help.

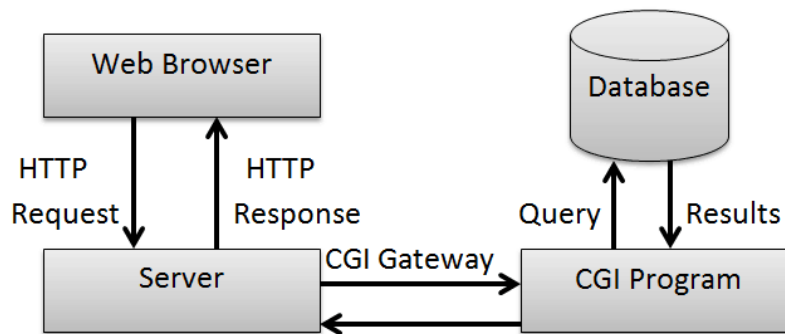


Figure 7 - Simplified CGI architecture

Rather than spending the rest of the summer blocked on the web interface, I looked for another challenge for the arm. While browsing online, I saw a video of an assembly line in which a robotic arm detected objects and moved them into place. I decided to try something similar.

The solution came from a component I already had on my desk: an HC-SR04 ultrasonic distance sensor that I had previously used in a gesture-activated lamp project. The sensor estimates the distance to an object by emitting ultrasonic waves and measuring the time taken for the echo to return. It has power and ground connections as well as TRIG and ECHO pins used to trigger a measurement and receive the reflected signal.

3.4.1 Distance-detection program

After connecting the sensor to the Raspberry Pi and selecting the appropriate resistors, I wrote the first distance-measurement program. The basic principle was simple: the program sent a trigger pulse, measured the time between emission and return, multiplied the time interval by the speed of sound, and divided the result by two because the sound wave travelled to the object and back.

I tested the program by placing the sensor next to a ruler and positioning an object at a known distance. The first results were disappointing because the measurements were not particularly accurate.

```

24     #settare metodo conteggio pin
25     GPIO.setmode(GPIO.BCM)
26     #settare il numero dei pin del sensore di distanza
27     TRIG = 23
28     ECHO = 18
29     #inizializzare i pin del settore di distanza
30     GPIO.setup(TRIG,GPIO.OUT)
31     GPIO.setup(ECHO,GPIO.IN)
32     #inizializzare il pin Trig
33     GPIO.output(TRIG, False)
34     time.sleep(1)
35     GPIO.output(TRIG, True)
36     time.sleep(0.00001)
37     GPIO.output(TRIG, False)
38     #segnare il momento in cui parte il suono
39     while GPIO.input(ECHO)==0:
40         pulse_start = time.time()
41     # segnare il momento di arrivo dell onda
42     while GPIO.input(ECHO)==1:
43         pulse_end = time.time()
44     # calcolare il tempo di viaggio del suono
45     pulse_duration = pulse_end - pulse_start
46     #calcolare la distanza percorsa
47     distance = pulse_duration * 17150
48     distance = round(distance, 2)
49     print("Distance:",distance,"cm")
50     return distance
51     GPIO.cleanup()
  
```

Figure 8 - First ultrasonic distance-measurement implementation

3.4.2 Distance-based movement

Once the sensor was working, I integrated the measurement code into the main program so that the arm could detect and collect an object placed on the work surface. This introduced several new problems. The first was a simple Python issue: I had to pass the measured distance from one function to another. With help from my classmate Danylo, I solved it by returning the calculated value from the detection function and assigning it to a variable in the movement function.

The movement routine first placed the arm in its starting position and then entered a loop that repeatedly called the distance function. If the detected object was more than 20 cm away, the base continued sweeping from side to side like a radar. When the measured distance dropped below 20 cm, the rotation stopped. In the first version, the arm also moved slightly forward because the detection point was not yet the ideal grabbing position.

```

67 def movement():
68     x = 0
69     #posizioni dei motori
70     b = 450
71     c = 100
72     d = 200
73     e = 150
74     f = 150
75     keep = True
76     #comunicare la gradazione a cui devono ruotare i motori
77     servo2.value= math.sin(math.radians(c))
78     servo3.value= math.sin(math.radians(d))
79     servo4.value= math.sin(math.radians(e))
80     servo5.value= math.sin(math.radians(f))
81     while keep:
82         #chiamare la distanza dalla funzione precedente
83         distance = clean()
84         #se la distanza è maggiore di 10 il motore ruota
85         if distance >= 20:
86             b += 5
87             servo1.value= math.sin(math.radians(b))
88
89         #se no si ferma la rotazione
90         elif distance < 20:
91             x = b + 10
92             servo1.value= math.sin(math.radians(x))
93
94             keep = False
95
96         if distance <20 and distance >= 15:
97             c = 166
98             d = 191
99             e = 105
100            f = 185

```

Figure 9 - Early distance-based object-search and grabbing logic

After the arm reached the correct position, the program selected predefined servo positions according to the measured distance. These values had been obtained experimentally: for several distance ranges, roughly every four centimetres, I manually positioned the arm with the keyboard until it could grab the object and recorded the corresponding values.

Although the approach was simple, the program initially failed after running for a while because the distance sensor stopped returning values. I spent a long time searching for the cause and eventually posted the problem on the Raspberry Pi forum. A user identified the likely conflict: the `GPIO.cleanup()` call from `RPi.GPIO` interfered with the library being used for servo control. Mixing the two GPIO-control approaches was causing the arm to stop working.

I therefore rewrote the sensor part using `gpiozero` as well. This was easier than expected because `gpiozero` already includes a `DistanceSensor` abstraction. Using the same pin-factory approach also improved the stability and accuracy of the measurements.

```

25 from gpiozero import DistanceSensor
26 echo_pin = 7
27 trig_pin = 23
28 sensor = DistanceSensor( echo = 7, trigger = 23, pin_factory = factory)
29

```

Figure 10 - gpiozero DistanceSensor configuration

```

158 def clean():
159     distance = round((sensor.distance* 100), 2)
160     print('Distance: ', distance, "cm")
161     time.sleep(0.01)
162     return distance

```

Figure 11 - Simplified distance-reading function

The movement function itself remained almost unchanged. I removed the additional forward movement after detection, and instead of accepting the first measurement immediately, the program took five more readings at 0.1-second intervals. This gave the arm time to settle and produced a more reliable distance estimate.

3.4.3 Trigonometry-based movement

Once the arm could detect and grab objects reliably, one of the more memorable accidents of the project happened. While testing the new program, I forgot to move the breadboard and its cables out of the arm's working area. The object I had placed on the desk was too far away for the sensor to detect, so the arm continued scanning until it encountered the breadboard. It then tried to "grab" it, caught the cables and pulled them out. The arm immediately collapsed because it had disconnected its own power. I then spent the following hours reconnecting everything.

More importantly, I was still not satisfied with the object-grabbing logic. The arm relied on a set of experimentally determined cases, so the joint angles were only defined for a limited number of distances. I wanted a formula that could calculate the required angles for any detected distance.

I realized that when the arm reached down toward an object, its geometry could be represented as a triangle: two sides were formed by the two arm segments, while the third side was the distance from the base to the object. This suggested using the law of cosines to calculate the required joint angles.

The program therefore defined two variables for the calculated angles, the fixed lengths of the two arm segments, and the measured distance between the sensor and the object. I added approximately three centimetres to the sensor reading because the sensor itself was mounted offset from the motor's rotational centre. The effective arm lengths also included the metal brackets connected to the servo shafts.

```

212 alpha = 0
213 beta = 0
214 arm1Lenght = 25
215 arm2Lenght = 32
216 distance1 = distance1 +3
217 alpha = math.acos((distance1**2+ arm1Lenght**2 -arm2Lenght**2)/(2*distance1*arm2Lenght))
218 beta = math.acos((arm2Lenght**2+ arm1Lenght**2 -distance1**2)/(2*arm1Lenght*arm2Lenght))

```

Figure 12 - Trigonometric calculation of the arm angles

With the three side lengths known, the program applied two forms of the law of cosines, rearranged to isolate the required angles. This produced the geometric angles, but I still needed to relate those angles to the actual servo commands. I therefore performed additional empirical tests to find the relationship between the triangle geometry and the motor positions.

```

220 servo2.value= math.sin(360 -math.radians(math.degrees(alpha)))
221 servo3.value= math.sin(math.radians(155 +math.degrees(beta)))

```

Figure 13 - Empirical mapping from calculated angles to servo positions

For the joint between the two arm segments, the correct motor position could be obtained by adding the calculated angle (converted to degrees) to a base value of approximately 155. The other angle was more difficult. During experimentation I accidentally mixed a value in radians with a degree-based offset and, unexpectedly, the resulting command worked in practice. I could not fully explain why this empirical relationship worked, so I left the functioning implementation unchanged - a practical example of the "if it works, do not touch it" rule that sometimes appears during prototyping.

3.5 Mark II gets a new look

By this stage, both major software routines were working, so I decided to improve the arm mechanically. One priority was to protect the breadboard and wiring so that the arm could not pull its own cables out again.

My first idea was to build a custom plywood enclosure. I bought the material, built the box and mounted the arm inside it, but the plywood could not support the weight and the structure collapsed. I therefore looked for something stronger and found a wooden wine box in the cellar. It was solid enough to support the arm, so I cut and modified it, added openings for the Raspberry Pi cables, and created a matching plywood platform on which objects could be placed.

I initially painted the box red on the outside and white on the inside, later repainting it black. I also painted the arm red and added two moving eyes to give the prototype more personality.



Figure 14 - Mark II mounted on its new base

3.5.1 The new look 2.0

At this point the report jumps from the end of the summer to around mid-November. During that period I continued working mainly on remote control, but I will finish the mechanical-design story first so that the topic remains together.

The next major upgrade came when I received a 3D printer for my birthday. After configuring it and printing a few test models, I decided to replace the wooden arm segments with 3D-printed parts. The goal was not only aesthetic:

by designing the pieces myself, I could incorporate external cable channels and eliminate the taped-on wires that frequently got in the way or became tangled.

After working through several tutorials on 3D modelling, I designed and printed the two replacement arm sections using Onshape.

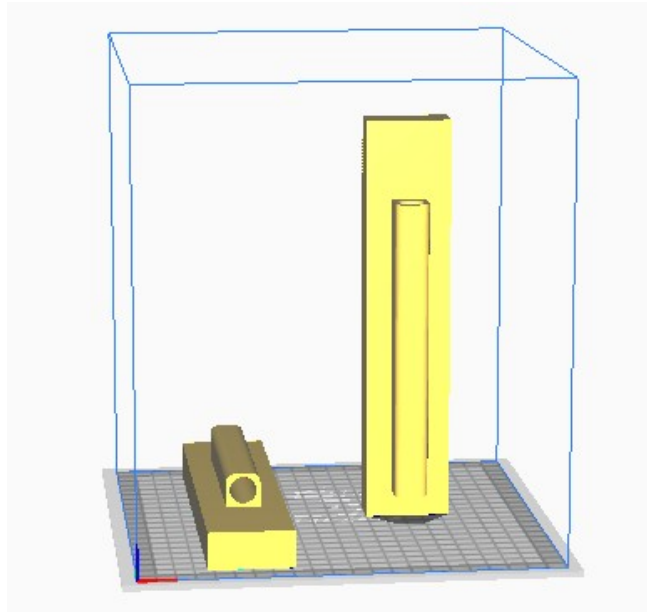


Figure 15 - 3D design of the replacement arm section

Installing the new parts was one of the most time-consuming stages. I had to remove all of the motors, reconnect them to the printed components, route each wire through the channels and insulate the connections with heat-shrink tubing. Some wires were too short and had to be extended. The work took an entire weekend.

Shortly afterwards, a single TRIG wire from the distance sensor shifted by one position on the breadboard. The mistake was difficult to spot because nothing was visibly broken. I spent about two and a half weeks checking the program and reconstructing the wiring before finally finding the misplaced connection.

I then redesigned the gripper. Until then, the gripping surfaces had effectively been LEGO pieces taped to the aluminium servo brackets. I used an existing gripper design from Thingiverse and created small additional 3D-printed cylinders to cover the joining screws. The two opposing gripper halves were mechanically linked so that the driven side moved the free side as well, increasing the effective gripping surface.

The eyes remained, and I also added a small 3D-printed hat. With the mechanical upgrades complete, this version became Mark III.



Figure 16 - Mark III after the 3D-printed mechanical upgrade

4 Remote control

4.1 Website control

I will keep this section relatively concise because, although it took a significant amount of time, the web-control approach never became as complete as I had hoped. The CGI-based solution was useful for learning how dynamic websites can communicate with Python programs, but it was limited for this application and the documentation I found was often incomplete.

With help from my supervisor, I eventually managed to make a Python file communicate with an HTML page on a small scale by using a button to switch an LED on and off. Even this simple example required a great deal of searching because most modern examples used different technologies.

The key command I found was `form.getvalue()`, which made it possible for the Python program to read the value of a submit button from the HTML form. The main difficulty was that submit buttons caused the page to reload after every action, and errors often resulted in a generic "INTERNAL SERVER ERROR". Only later did I discover where Apache stored the relevant error information.

Once the basic communication worked, I replaced the LED command with the servo-movement commands from the keyboard-control program and added two buttons for each motor. Most changes on the HTML side were visual, including styling the buttons and adding a background image.

```

35 print( "Content-type: text/html\n")
36 form = cgi.FieldStorage()
37 #check wich button is pressed and store the new servo position
38 if form.getvalue("sinistra1"):
39     b += speed
40     if b > 440 :
41         b = 440
42 if form.getvalue("destra1"):
43     b -= speed
44     if b < 290:
45         b = 290

```

Figure 17 - Python CGI code used for web-based motor commands

4.1.1 Saving the arm position

The most interesting part of the website program was the way motor positions were stored. Because the CGI Python script restarted every time a button was pressed, the program could not keep the updated servo positions in memory between requests. I therefore needed an external way to save the state and reload it the next time the script executed.

After several attempts, and with help from my supervisor, I used a text file. This allowed the program to write the current motor positions externally and read them again on the next execution. The same state-storage mechanism later became useful in the SSH-based control system.

At this stage the local website technically worked: from a device on the same network, I could navigate to the Raspberry Pi's IP address and control the arm with buttons. The main problem was movement quality. Each button press moved a servo by a relatively large step, which made the arm oscillate. CGI processing also introduced noticeable delay, especially when buttons were pressed in quick succession.

I attempted a different interaction model in which one button press would start continuous movement and a second press would stop it. I never managed to make that approach reliable within the CGI design, mainly because the submit-button behaviour kept forcing page reloads. I therefore decided to try a different remote-control method: SSH.

4.2 SSH connection

The purpose of the SSH solution was to implement the continuous movement idea more directly. SSH (Secure Shell) is a network protocol that allows a user to access and execute commands on another computer securely over a network. I used it to remotely launch Python movement scripts on the Raspberry Pi.

On iOS, I used an SSH remote-control application that allowed me to create custom SSH buttons. Each button executed a predefined command on the Raspberry Pi. Before implementing movement, I first tested the concept with a small program that created a file and wrote "hello" into it. Once that worked, I moved on to motor control.

The SSH movement programs combined ideas from both the keyboard program and the web-server program. Like the keyboard controller, each script used a loop to increment or decrement a movement variable. Unlike the keyboard program, however, there was no single program controlling every motor. Instead, each direction of each motor had its own script, effectively corresponding to one remote-control button.

Because multiple independent scripts were involved, they needed a shared representation of the current motor positions. I reused the text-file approach. My first implementation opened the file and read fixed character ranges from individual lines, converting the resulting strings into integers.

```

1  dati = open('dati.txt', 'r')
2  b = int(dati.readline(123))
3  c = int(dati.readline(456))
4  d = int(dati.readline(789))
5  e = int(dati.readline(101112))
6  f = int(dati.readline(131415))
7  dati.close()

```

Figure 18 - First attempt at reading saved motor positions from a text file

This occasionally produced a ValueError because an empty line was being read and then converted to an integer. The issue became clear only after printing the data during debugging. I rewrote the routine so that it built a list containing only non-empty lines and then converted those values to integers.

```

17 file = open('/usr/lib/cgi-bin/dati.txt')
18 A = []
19 for i in file:
20     A.append(int(i.split("\n")[0]))
21 b = A[0]
22 c = A[1]
23 d = A[2]
24 e = A[3]
25 f = A[4]
26 file.close()

```

Figure 19 - Revised state-file parsing that ignores empty lines

With the current motor positions loaded, the movement script could update a servo and then write the new value back to the text file. The next challenge was stopping the loop from another program.

```

48 while stato == 1:
49
50     b += speed
51     time.sleep(0.1)
52     c = c
53     d = d
54     e = e
55     f = f
56     print(b)
57     if b > 440 :
58         b = 440
59         print("massimo raggiunto")
60     servo1.value= math.sin(math.radians(b))
61     servo2.value= math.sin(math.radians(c))
62     servo3.value= math.sin(math.radians(d))
63     servo4.value= math.sin(math.radians(e))
64     servo5.value= math.sin(math.radians(f))
65     file = open('/usr/lib/cgi-bin/dati.txt')
66     A = []
67     for i in file:
68         A.append(int(i.split("\n")[0]))
69     stato = A[5]
70     file.close()
71     dati = open('/usr/lib/cgi-bin/dati.txt','w')
72     dati.write("%s\n" % str(b))
73     dati.write("%s\n" % str(c))
74     dati.write("%s\n" % str(d))
75     dati.write("%s\n" % str(e))
76     dati.write("%s\n" % str(f))
77     dati.write("%s\n" % str(stato))
78
79     dati.close()

```

Figure 20 - Movement loop with state persistence

I solved this by introducing a shared state variable stored in the same text file. The movement loop continued only while this variable was equal to 1. At every cycle, the program reread the file and checked the current state. A separate stop script changed the state to 0, causing the active movement loop to terminate.

```

31 stato = 0
32 dati.write("%s\n" % str(stato))

```

Figure 21 - Stop command writing the shared state variable

```

46 while True:
47     b += speed
48     time.sleep(0.1)
49     c = c
50     d = d
51     e = e
52     f = f
53     if b > 440 :
54         b = 440
55         print("massimo raggiunto")
56     servo1.value= math.sin(math.radians(b))
57     servo2.value= math.sin(math.radians(c))
58     servo3.value= math.sin(math.radians(d))
59     servo4.value= math.sin(math.radians(e))
60     servo5.value= math.sin(math.radians(f))
61     dati = open('/usr/lib/cgi-bin/dati.txt', 'w')
62     dati.write("%s\n" % str(b))
63     dati.write("%s\n" % str(c))
64     dati.write("%s\n" % str(d))
65     dati.write("%s\n" % str(e))
66     dati.write("%s\n" % str(f))
67     dati.write("%s\n" % str(stato))
68     dati.close()
69

```

Figure 22 - Continuous movement controlled by the shared stop state

This introduced one final logical issue: after the stop state had been set to 0, a new movement script also needed to restore it to 1 before starting its own loop. I added that initialization to every movement script. With eight movement scripts plus the stop command, I could finally control all of the required directions remotely.

There was still one costly accident along the way. I accidentally launched the wrong program and the arm drove forcefully into its base, breaking the thin walls of the 3D-printed arm sections. I reprinted both pieces with walls roughly three times thicker and then had to remove, reroute and reconnect all of the wiring once again.

The final phone interface used SSH buttons configured with a name, the Raspberry Pi's IP address, SSH port, username, password and the command to execute. The command itself followed the normal pattern for running a Python file, for example: `python3 program_name.py`.

Figure 23 - SSH button configuration on the phone

After creating buttons for each movement direction and a dedicated stop command, the interface provided a practical remote controller. Pressing a movement button started the corresponding motor in one direction; pressing Stop ended the movement. The final result was not exactly the browser-based controller I had originally imagined, but the goal of remotely controlling the robotic arm had been achieved.



Figure 24 - Final remote-control screen

5 Conclusions

Building this project was, for the most part, an enjoyable experience and it taught me a great deal. I clearly improved my programming skills, but I also learned a new way of approaching robotics: using a Raspberry Pi to connect software directly to motors, sensors and physical mechanisms.

I also gained an introduction to web hosting and dynamic web pages, and I learned how to interact with a computer remotely through a protocol that had previously been unfamiliar to me: Secure Shell.

There were certainly moments when I wanted to abandon the whole thing. Many of the errors described in this report took hours or days to identify, and some problems returned after apparently being solved. At times I spent a week rebuilding connections simply to understand why something had stopped working, or trying to understand why the same code worked once and then failed on the next run.

Overall, however, it was a very valuable experience, and I was satisfied with the final result. More than the finished arm itself, the project taught me how much iteration, troubleshooting and practical experimentation are involved when software has to control a real physical system.

6 Bibliography

[https://it.wikipedia.org/wiki/Mark_\(modello\)](https://it.wikipedia.org/wiki/Mark_(modello))

<https://en.wikipedia.org/wiki/Servomotor>

https://www.researchgate.net/figure/Schematic-of-an-RC-servo-motor_fig1_328910852

<https://www.aliexpress.com/item/32907625266.html>

<https://pypi.org/project/RPi.GPIO/>

<https://www.pygame.org/>

<https://gpiozero.readthedocs.io/en/stable/>

<https://networkencyclopedia.com/common-gateway-interface-cgi/>
<https://forums.raspberrypi.com/>
<https://www.onshape.com/>
<https://www.thingiverse.com/thing:3648782>
<https://www.techtarget.com/searchsecurity/definition/Secure-Shell>

Translation note: This English edition is based on the original 2021-2022 Italian Matura report. The original project chronology, technical decisions, setbacks and outcomes are preserved; phrasing has been lightly edited to improve clarity and readability in English.